

LARA: Training Data Generation Appendix

Alessandro Berti

Wil M. P. van der Aalst

A Training Data Generation

This standalone appendix expands the training data generation description in the main paper. The data generator creates a supervised alignment corpus in which every row contains a Petri net, one trace, and the exact optimal alignment for that trace on that net. The relevant code path is `scripts/init_data.py`, which calls the behavior-family generator in `lara_align/families.py`, the PM4Py exact aligner in `lara_align/exact.py`, and the replay verifier in `lara_align/verify.py`. The checked-in reference corpus uses dataset format version 2 and the defaults represented in `configs/behavior_families.json`.

A.1 What One Training Row Contains

One generated row is an `AlignmentSample`. It contains:

- a unique sample id and its split;
- a Petri net SN with its initial and final markings;
- one observed trace σ ;
- an optimal alignment γ^* computed by the exact backend;
- the optimal cost $\delta = c(\gamma^*)$ under the standard move costs;
- metadata describing how the row was generated.

The standard costs are the same costs used throughout the paper: synchronous moves cost zero, visible log moves cost one, visible model moves cost one, and silent model moves cost zero. The exact alignment is transition-aware. Thus a synchronous move stores the concrete transition name, not only the visible activity label. This matters when two transitions have the same label.

The neural training targets are derived from this exact alignment. For each trace event, the sync target is the transition fired by the exact alignment when that event is synchronized. Events skipped by the optimum become log-move targets. Transitions fired without consuming an event become model-move targets. The exact cost is also stored as the target for the learned cost estimate.

A.2 Behavior Families

The generator does not create isolated trace and net pairs directly. It first creates a *behavior family*. A behavior family is a small group of related models and traces that represent the same visible behavior in different Petri net forms. This is the unit that is assigned to train, validation, or test before any row is written. Assigning whole families to a split prevents a model variant from one behavior appearing in training while another variant of the same behavior appears in testing.

Table 1: Behavior-family motifs used in the reference corpus. Each family contains two Petri net representations of the same visible behavior.

motif	representation 1	representation 2
ordinary tree	canonical block net	isomorphic renaming
duplicate versus silent	duplicate-prefix net	silent-routing net
concurrent versus interleaved	parallel net	explicit interleaving net
M-pattern	canonical block net	non-free-choice M-pattern net

With the default configuration, each family has two Petri net representations and two observed traces. Therefore each family expands to four rows:

$$2 \text{ representations} \times 2 \text{ traces} = 4 \text{ rows.}$$

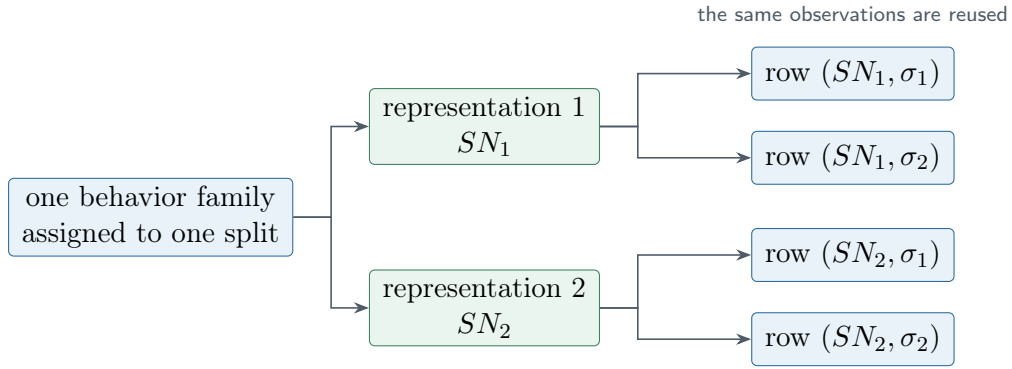


Figure 1: Expansion of one behavior family into four supervised rows. The split is chosen at family level, and both representations receive the same two observed traces.

The reference corpus uses four family motifs with equal weight. Table 1 summarizes them.

Before generation, the requested family count is converted into a deterministic motif quota. The quota reserves the configured per-motif minimum, distributes the remaining families according to the motif weights, and shuffles the plan with a split-specific seed. If an attempted family is rejected, the next family index is tried for the same planned motif. Consequently, rejection does not silently change the requested class balance. The family id, rather than an individual row id, is the split key.

Ordinary Tree. The ordinary-tree motif samples a random block-structured process tree. The default tree has between 4 and 12 visible activity occurrences and depth at most 6. Internal nodes are sampled from sequence, exclusive choice, and parallel composition with weights 0.4, 0.3, and 0.3. The visible alphabet size is about 70% of the number of visible occurrences, so duplicate labels can appear naturally. The first representation is the compiled block-structured Petri net. The second representation is an isomorphic renaming of the same net: places and transition names are changed, but labels and behavior are preserved.

Figure 2 uses a small sampled tree to illustrate the transformation. In the Petri net figures, circles are places, blue rectangles are visible transitions, dark rectangles are silent transitions, and a bullet marks the initial token.

Duplicate Versus Silent. This motif has two ways to represent the visible behavior “A followed by either B or C”. In the duplicate-prefix representation, there are two different visible transitions labeled

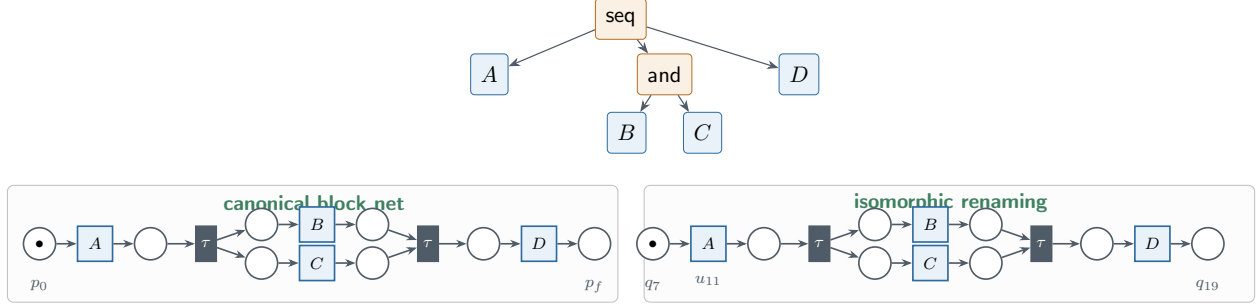


Figure 2: Illustrative ordinary-tree family for $\text{seq}(A, \text{and}(B, C), D)$. The second net preserves all arcs and visible labels while changing place and transition identities. Both nets have the complete visible language $\{\langle A, B, C, D \rangle, \langle A, C, B, D \rangle\}$.

A : one leads to the B branch and the other to the C branch. In the silent-routing representation, there is one visible A transition followed by a silent choice between the two branches. This motif directly tests whether the model learns transition identity rather than only label identity.

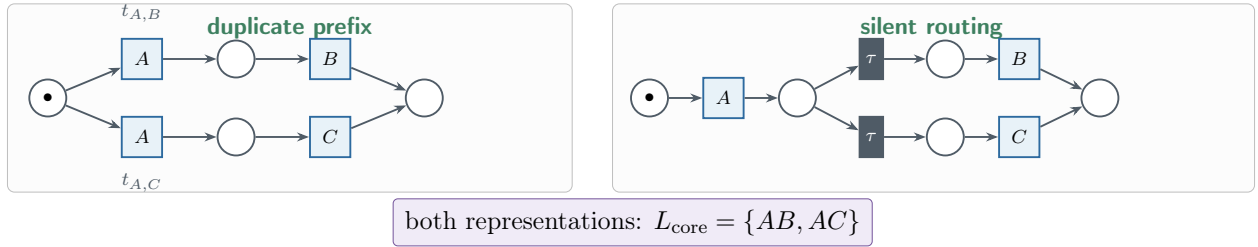


Figure 3: Two representations of the duplicate-versus-silent motif. The left net contains two concrete transitions with label A ; the right net contains one A transition and a silent choice.

Concurrent Versus Interleaved. This motif represents the visible language $\langle A, B \rangle$ and $\langle B, A \rangle$ in two different ways. One representation uses real parallelism: an invisible split produces two concurrent branches, and an invisible join closes them. The other representation encodes the two possible orders explicitly as alternatives. The two nets have the same visible complete traces but different causal structure.

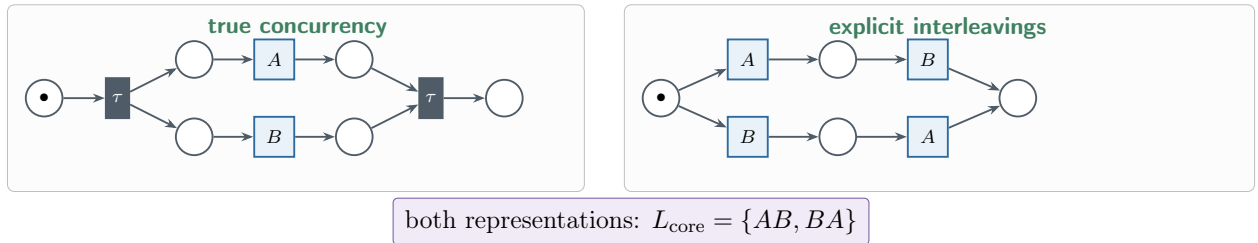


Figure 4: Two representations of the concurrent-versus-interleaved motif. Independent enabled transitions produce both orders on the left; an exclusive choice between two fixed paths produces the same visible language on the right.

M-pattern. This motif compares a block-structured net with a non-free-choice M-pattern. The visible behavior is an exclusive choice between B and the parallel execution of A and C . The non-free-choice representation shares input places in a way that violates free choice. This gives the model examples whose behavior is simple but whose Petri net structure is not block structured.

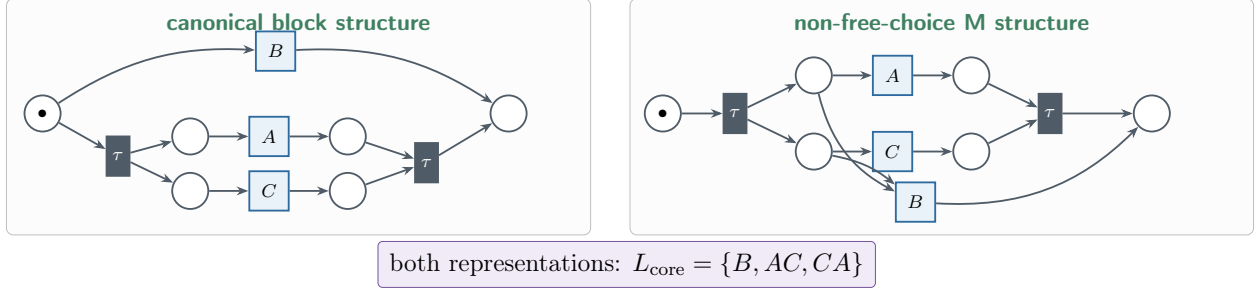


Figure 5: Two representations of the M-pattern motif. In the right net, A and B share one input place while B and C share another, so the choice is not free. The complete visible languages still agree.

For the three controlled motifs above, the default generator appends a short shared suffix of two fresh visible activities. This makes the examples less degenerate while preserving the intended comparison between the two representations.

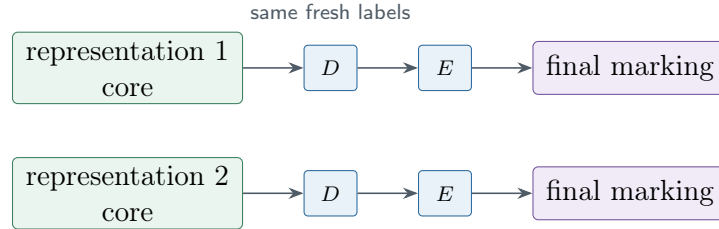


Figure 6: Shared visible suffix appended to both representations of each controlled motif. The suffix changes the full traces but not the comparison between representation structures.

A.3 Checking That Representations Match

After the representations are built, the generator enumerates their visible complete-trace languages. The default limits for this check are 5,000 search states, 10,000 traces, and visible length 20; a separate firing bound prevents unbounded silent exploration. The certificate is **exact** only when the enumeration finishes without hitting a bound, and is otherwise **bounded**. A family is rejected if the enumerated languages disagree. A family is also rejected if the reference language is empty, the final marking is unreachable, or any transition is dead. By default, bounded certificates are additionally rejected for training; presets intended for bounded loops can relax that requirement.

The reference corpus is small enough that all accepted train, validation, and test rows have an exact visible-language certificate. In other words, the language comparison completed within the limits for every accepted row.

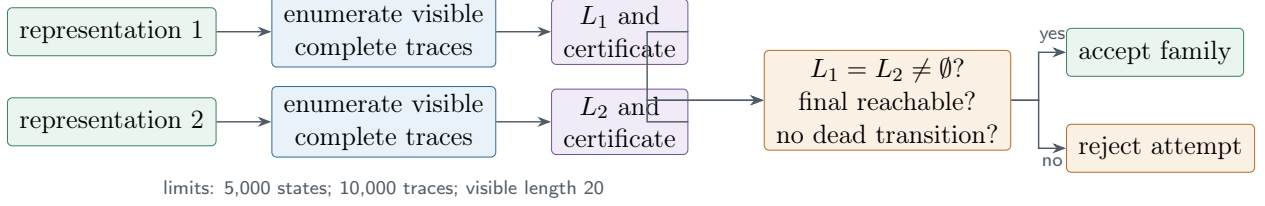


Figure 7: Family-level representation check. Enumeration records whether each language result is exact or bounded; default training additionally requires an exact certificate. A rejected attempt is replaced without changing its planned motif.

A.4 Clean Trace Pool

For each accepted family, the generator sorts the enumerated visible language and keeps at most 16 clean traces. If the language has more than 16 traces, a deterministic sample of 16 indices is selected with the family model RNG and then restored to language order. These clean traces are not training rows yet. They are the fitting traces from which observed traces are produced.

The default configuration creates two observed traces per family. The first two clean traces in the selected pool are used, wrapping around if the pool is smaller than two. Trace generation is deterministic given the global seed, the split name, and the family index.

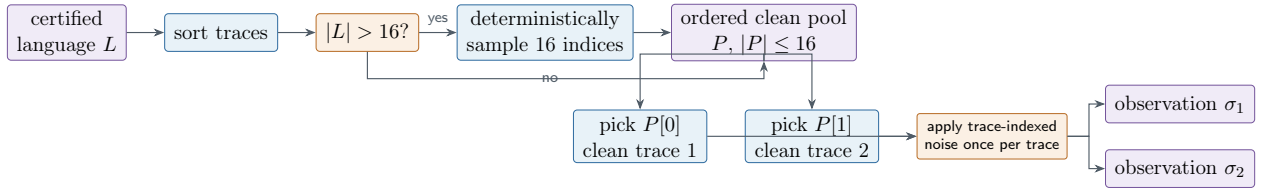


Figure 8: Construction of the two observations for an accepted family. A small pool is selected from the certified language, then each selected clean trace is corrupted once. The resulting σ_1 and σ_2 are reused for both representations.

A.5 Trace Noise

Noise is added once at the visible-label level. The same noisy trace is then paired with every representation in the family. This is important: if a family has a duplicate-prefix net and a silent-routing net, both nets see the same observed trace. The exact aligner then decides how that same observation should be explained on each representation.

With the default noise settings, 20% of observed traces are kept clean. The remaining traces receive one, two, or three edits with probabilities 0.5, 0.3, and 0.2. The edit operations are:

- delete one event;
- insert an event whose label is already in the family alphabet;
- substitute one event label by another label from the family alphabet;
- swap two neighboring events;
- repeat one event;

- remove a prefix;
- remove a suffix;
- insert the special outside label `--OOD--`.

The first five operations have weight 0.15 each, prefix and suffix truncation have weight 0.10 each, and outside insertion has weight 0.05. Generated traces are capped at length 128. An operation is drawn independently at every edit. If the selected operation is inapplicable to the current trace (for example, a deletion from an empty trace or a swap in a one-event trace), the implementation falls back to outside insertion, so every requested edit still produces a recorded operation. Consequently, the realized outside-insertion frequency can be higher than its nominal weight on very short traces.

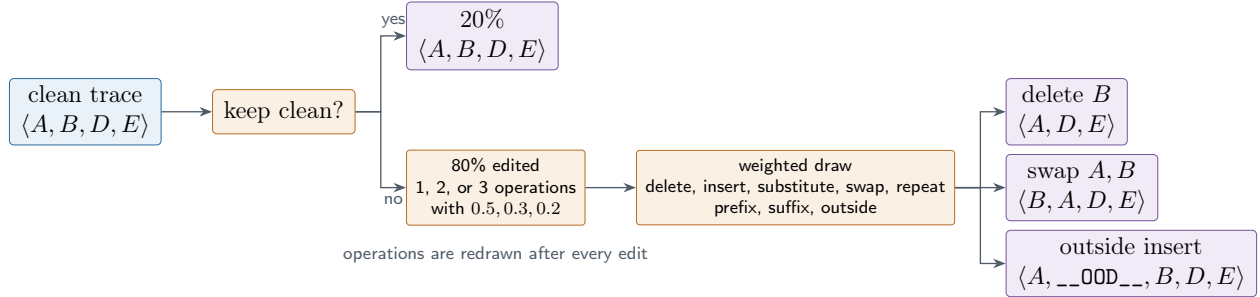


Figure 9: Default trace-corruption mechanism with three illustrative outcomes. The examples show individual edits; multi-edit observations repeatedly apply the same weighted operation draw.

A.6 Exact Labeling

The initializer expands every accepted family into trace and net pairs and sends each pair to the PM4Py state-equation A* aligner. The reference generation uses no per-trace timeout. The aligner is run with transition-aware output enabled, so the returned alignment identifies the actual Petri net transition used by each synchronous or model move. Exact results are cached by net signature, observed label sequence, and cost-model version within each split.

Every exact result is replay-checked by LARA’s verifier. Replay checking starts from the initial marking, fires the model side of the alignment move by move, checks that the log projection equals the observed trace, and checks that the final marking is reached. If replay fails, the row is rejected. If PM4Py does not return an optimal alignment, the whole family is rejected and another family index is tried. Rows are buffered until every trace and representation pair in the family passes, so no partial family is written under the default strict coverage mode.

For families whose representation equivalence was certified exactly, the initializer also checks cost consistency: the same observed trace must have the same optimal cost on all equivalent representations. If equivalent representations disagree in cost, the family is rejected. This prevents the model from receiving contradictory targets for what is meant to be the same visible behavior.

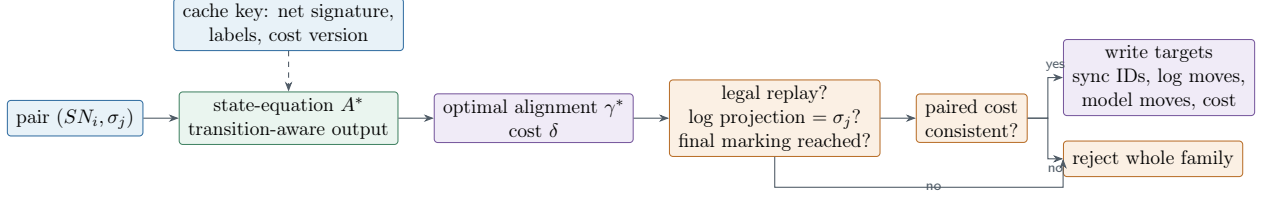


Figure 10: Exact-labeling and verification pipeline. Rows are committed only after every pair in the family passes replay; exactly certified equivalent representations must also agree on optimal cost.

A.7 Masked Transition-Identity Targets

Most synchronous targets can be used directly. One corner case occurs in the duplicate-prefix representation. Its A transition is considered identifiable exactly when the observed labels contain B or C , but not both. Noise can remove both distinguishing labels or introduce both, in which case a particular optimal A witness is not an identifiable target. The implementation then masks all synchronous transition-identity targets for that row from the sync loss. The row is still kept: log-move targets, model-move targets, legality, and optimal-cost supervision remain valid. The metadata records this as `transition_identity_loss_masked`.

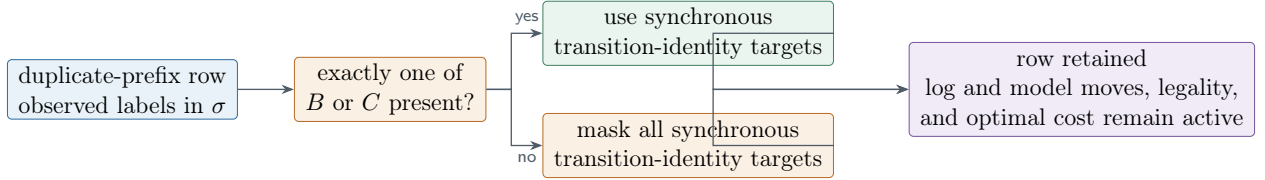


Figure 11: Masking rule for ambiguous duplicate-prefix observations. Masking removes only transition-identity supervision, not the row or its other exact targets.

A.8 Reference Split Sizes

The default command

```
python scripts/init_data.py -overwrite
```

creates the reference corpus in `data/lara_synthetic`. It writes one pickle file per split, plus machine-readable and human-readable metadata:

- `train.pkl`, `val.pkl`, and `test.pkl`;
- `metadata.pkl` and `metadata.json`.

Table 2: Reference corpus generated with seed 13. Each family expands to four rows because it has two representations and two observed traces.

split	families	rows	families per motif
train	512	2,048	128
validation	128	512	32
test	128	512	32

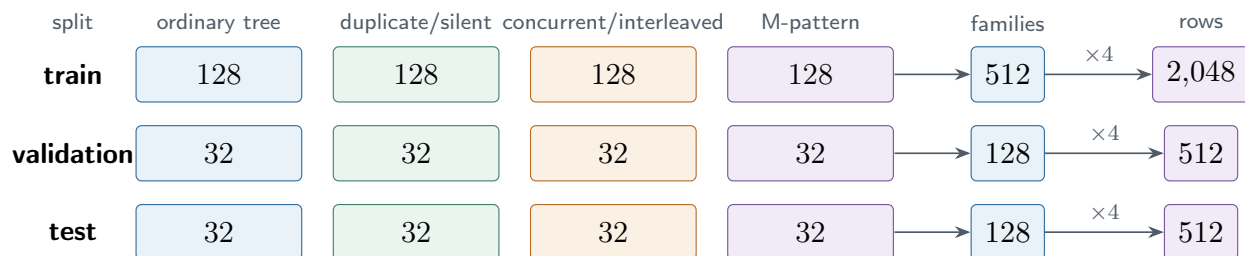


Figure 12: Balanced family quotas and row expansion in the reference corpus. Every motif remains wholly inside its assigned split, and every accepted family contributes four rows.

Strict class coverage is used by default. This means the requested split size must contain complete families, and every active motif must meet its required minimum. In the reference corpus all four motifs are exactly balanced in every split. The manifest records one generation detail that does not affect the accepted corpus: 25 attempted families raised a generation-time **ValueError** and were replaced by later accepted families of the same planned motifs. No exact-alignment, replay, or paired-cost rejection was recorded for this run.